

Objective

The purpose of this lab is to explore model selection, evaluation, and hyperparameter tuning using two widely-used real-world datasets.

- Compare the performance of a naive train-test split with k-fold cross-validation.
- Implement hyperparameter tuning using GridSearchCV (or RandomizedSearchCV) for Logistic Regression (binary classification) and Linear Regression (regression).
- Compare evaluation metrics: Accuracy and F1 Score for classification; Mean Squared Error (MSE) and R2 for regression.
- Display visualizations directly in the document for all comparisons.

Dataset 1: Breast Cancer Wisconsin (Diagnostic) Dataset

- Load the dataset
- Handle missing values if present.
- Split the dataset into features (X) and binary target labels (y).
- Normalize or standardize the features.

```
# Load and Preprocess Data  
from sklearn.datasets import load_breast_cancer  
data_class = load_breast_cancer()  
X_class = data_class.data  
y_class = data_class.target
```

```
# Task 1: Train-Test Split vs. Cross-Validation
# a. Naive Train-Test Split
from sklearn.model_selection import train_test_split
X_train_class, X_test_class, y_train_class, y_test_class = train_test_split(X_class, y_class, test_size=0.3, random_state=42)
```

```
from sklearn.preprocessing import StandardScaler
class_scaler = StandardScaler()
X_train_class_scaled = class_scaler.fit_transform(X_train_class)
X_test_class_scaled = class_scaler.transform(X_test_class)
```

Dataset 2: Breast Cancer Wisconsin (Diagnostic) Dataset

- Load the dataset
- Handle missing values if present.
- Split the dataset into features (X) and the continuous target variable (y).
- Apply feature scaling as needed.

```
from sklearn.datasets import fetch_california_housing
data = fetch_california_housing()
X_reg = data.data
y_reg = data.target
```

```
from sklearn.model_selection import train_test_split
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(X_reg, y_reg, test_size=0.3, random_state=42)
```

```
reg_scaler = StandardScaler()
X_train_reg_scaled = reg_scaler.fit_transform(X_train_reg)
X_test_reg_scaled = reg_scaler.transform(X_test_reg)

reg_y_scaler = StandardScaler()
y_train_reg_scaled = reg_y_scaler.fit_transform(y_train_reg)
y_test_reg_scaled = reg_y_scaler.transform(y_test_reg)
```

```
from sklearn.model_selection import train_test_split
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(X_reg, y_reg, test_size=0.3, random_state=42)
```

```
reg_scaler = StandardScaler()
X_train_reg_scaled = reg_scaler.fit_transform(X_train_reg)
X_test_reg_scaled = reg_scaler.transform(X_test_reg)

reg_y_scaler = StandardScaler()
y_train_reg_scaled = reg_y_scaler.fit_transform(y_train_reg)
y_test_reg_scaled = reg_y_scaler.transform(y_test_reg)
```

```
File ~\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2954, in validate_data(_estimator, X, y, reset, validate_separately, skip_check_array, **check_params)
```

```
2952         out = X, y
2953     elif not no_val_X and no_val_y:
-> 2954         out = check_array(X, input_name="X", **check_params)
2955     elif no_val_X and not no_val_y:
2956         out = _check_y(y, **check_params)
```

```
File ~\anaconda3\Lib\site-packages\sklearn\utils\validation.py:1091, in check_array(array, accept_sparse, accept_large_sparse, dtype, order, copy, force_writeable, force_all_finite, ensure_all_finite, ensure_non_negative, ensure_2d, allow_nd, ensure_min_samples, ensure_min_features, estimator, input_name)
```

```
1084         else:
1085             msg = (
1086                 f"Expected 2D array, got 1D array instead:\narray={array}.\n"
1087                 "Reshape your data either using array.reshape(-1, 1) if "
1088                 "your data has a single feature or array.reshape(1, -1) "
1089                 "if it contains a single sample."
1090             )
-> 1091         raise ValueError(msg)
1093     if dtype_numeric and hasattr(array.dtype, "kind") and array.dtype.kind in "USV":
1094         raise ValueError(
1095             "dtype='numeric' is not compatible with arrays of bytes/strings."
1096             "Convert your data to numeric values explicitly instead."
1097         )
```

ValueError: Expected 2D array, got 1D array instead:

array=[1.938 1.697 2.598 ... 2.221 2.835 3.25].

Reshape your data either using array.reshape(-1, 1) if your data has a single feature or array.reshape(1, -1) if it contains a single sample.

```
File ~\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2954, in validate_data(_estimator, X, y, reset, validate_separately, skip_check_array, **check_params)
```

```
2952         out = X, y
2953     elif not no_val_X and no_val_y:
-> 2954         out = check_array(X, input_name="X", **check_params)
2955     elif no_val_X and not no_val_y:
2956         out = _check_y(y, **check_params)
```

```
File ~\anaconda3\Lib\site-packages\sklearn\utils\validation.py:1091, in check_array(array, accept_sparse, accept_large_sparse, dtype, order, copy, force_writeable, force_all_finite, ensure_all_finite, ensure_non_negative, ensure_2d, allow_nd, ensure_min_samples, ensure_min_features, estimator, input_name)
```

```
1084         else:
1085             msg = (
1086                 f"Expected 2D array, got 1D array instead:\narray={array}.\n"
1087                 "Reshape your data either using array.reshape(-1, 1) if "
1088                 "your data has a single feature or array.reshape(1, -1) "
1089                 "if it contains a single sample."
1090             )
-> 1091         raise ValueError(msg)
1093     if dtype_numeric and hasattr(array.dtype, "kind") and array.dtype.kind in "USV":
1094         raise ValueError(
1095             "dtype='numeric' is not compatible with arrays of bytes/strings."
1096             "Convert your data to numeric values explicitly instead."
1097         )
```

```
ValueError: Expected 2D array, got 1D array instead:
array=[1.938 1.697 2.598 ... 2.221 2.835 3.25 ].
Reshape your data either using array.reshape(-1, 1) if your data has a single feature or array.reshape(1, -1) if it contains a single sample.
```

```
reg_y_scaler = StandardScaler()  
y_train_reg_scaled = reg_y_scaler.fit_transform(y_train_reg.reshape(-1,1))  
y_test_reg_scaled = reg_y_scaler.transform(y_test_reg.reshape(-1,1))
```

Task 1: Evaluation with Train-Test Split vs. Cross-Validation

a. Naive Train-Test Split:

- Split each dataset into a training set (e.g., 70%) and a testing set (e.g., 30%).
- Train the model:
 - For the Breast Cancer Dataset: Use Logistic Regression.
 - For the California Housing Dataset: Use Linear Regression.
- Evaluate the models on the testing set using:
 - For classification: Accuracy and F1 Score.
 - For regression: Mean Squared Error (MSE) and R^2 .

```
from sklearn.linear_model import LogisticRegression, LinearRegression #, Ridge, Lasso
from sklearn.metrics import accuracy_score, f1_score, mean_squared_error, r2_score
```

```
# Logistic Regression for Breast Cancer Dataset
```

```
log_reg = LogisticRegression(max_iter=10000)
```

```
log_reg.fit(X_train_class_scaled, y_train_class)
```

```
y_pred_class = log_reg.predict(X_test_class_scaled)
```

```
# Evaluation Metrics for train-test split
```

```
acc = accuracy_score(y_test_class, y_pred_class)
```

```
f1 = f1_score(y_test_class, y_pred_class)
```

```
print("accuracy is ", acc)
```

```
print("F1 score is ", f1)
```

```
accuracy is 0.9824561403508771
```

```
F1 score is 0.986046511627907
```

```
# Linear Regression for California Housing Dataset
lin_reg = LinearRegression()
lin_reg.fit(X_train_reg_scaled, y_train_reg_scaled)
y_pred_reg = lin_reg.predict(X_test_reg_scaled)

mse = mean_squared_error(y_test_reg_scaled, y_pred_reg)
r2 = r2_score(y_test_reg, y_pred_reg)
print("mse is ", mse)
print("r2 is ", r2)
```

```
mse is  0.3960359599480819
r2 is  -2.670501046711584
```

```
# Linear Regression for California Housing Dataset
lin_reg = LinearRegression()
lin_reg.fit(X_train_reg_scaled, y_train_reg_scaled)
y_pred_reg = lin_reg.predict(X_test_reg_scaled)

mse = mean_squared_error(y_test_reg_scaled, y_pred_reg)
r2 = r2_score(y_test_reg, y_pred_reg)
print("mse is ", mse)
print("r2 is ", r2)
```

```
mse is  0.3960359599480819
r2 is  -2.670501046711584
```



```
# Linear Regression for California Housing Dataset
lin_reg = LinearRegression()
lin_reg.fit(X_train_reg_scaled, y_train_reg_scaled)
y_pred_reg = lin_reg.predict(X_test_reg_scaled)

mse = mean_squared_error(y_test_reg_scaled, y_pred_reg)
r2 = r2_score(y_test_reg, y_pred_reg)
print("mse is ", mse)
print("r2 is ", r2)
```

```
mse is  0.3960359599480819
r2 is  -2.670501046711584
```



```
reg_y_scaler = StandardScaler()
y_train_reg_scaled = reg_y_scaler.fit_transform(y_train_reg.reshape(-1,1))
y_test_reg_scaled = reg_y_scaler.transform(y_test_reg.reshape(-1,1))
```

```
# Linear Regression for California Housing Dataset
lin_reg = LinearRegression()
lin_reg.fit(X_train_reg_scaled, y_train_reg_scaled)
y_pred_reg = lin_reg.predict(X_test_reg_scaled)

y_pred_reg = reg_y_scaler.inverse_transform(y_pred_reg.reshape(-1,1));
y_test_reg = reg_y_scaler.inverse_transform(y_test_reg_scaled.reshape(-1,1));
mse = mean_squared_error(y_test_reg, y_pred_reg)
r2 = r2_score(y_test_reg, y_pred_reg)
print("mse is ", mse)
print("r2 is ", r2)
```

```
mse is  0.5305677824766754
r2 is  0.5957702326061662
```

```
reg_y_scaler = StandardScaler()
y_train_reg_scaled = reg_y_scaler.fit_transform(y_train_reg.reshape(-1,1))
y_test_reg_scaled = reg_y_scaler.transform(y_test_reg.reshape(-1,1))
```

```
# Linear Regression for California Housing Dataset
```

```
lin_reg = LinearRegression()
lin_reg.fit(X_train_reg_scaled, y_train_reg_scaled)
y_pred_reg = lin_reg.predict(X_test_reg_scaled)

y_pred_reg = reg_y_scaler.inverse_transform(y_pred_reg.reshape(-1,1));
y_test_reg = reg_y_scaler.inverse_transform(y_test_reg_scaled.reshape(-1,1));
mse = mean_squared_error(y_test_reg, y_pred_reg)
r2 = r2_score(y_test_reg, y_pred_reg)
print("mse is ", mse)
print("r2 is ", r2)
```

```
mse is 0.5305677824766754
```

```
r2 is 0.5957702326061662
```

```
reg_y_scaler = StandardScaler()
y_train_reg_scaled = reg_y_scaler.fit_transform(y_train_reg.reshape(-1,1))
y_test_reg_scaled = reg_y_scaler.transform(y_test_reg.reshape(-1,1))
```

Linear Regression for California Housing Dataset

```
lin_reg = LinearRegression()
lin_reg.fit(X_train_reg_scaled, y_train_reg_scaled)
y_pred_reg = lin_reg.predict(X_test_reg_scaled)
```

```
y_pred_reg = reg_y_scaler.inverse_transform(y_pred_reg.reshape(-1,1));
y_test_reg = reg_y_scaler.inverse_transform(y_test_reg_scaled.reshape(-1,1));
mse = mean_squared_error(y_test_reg, y_pred_reg)
r2 = r2_score(y_test_reg, y_pred_reg)
print("mse is ", mse)
print("r2 is ", r2)
```

mse is 0.5305677824766754

r2 is 0.5957702326061662

b. k -Fold Cross-Validation:

- Apply k -fold cross-validation (suggest $k = 5$ or $k = 10$) using `cross_val_score`.
- Report the mean and standard deviation of the evaluation metrics.

```
# b.  $k$ -Fold Cross-Validation
```

```
cv_acc_scores = cross_val_score(log_reg, X_class, y_class, cv=5, scoring='accuracy')
cv_f1_scores = cross_val_score(log_reg, X_class, y_class, cv=5, scoring='f1')
cv_mse_scores = -cross_val_score(lin_reg, X_reg, y_reg, cv=5, scoring='neg_mean_squared_error')
cv_r2_scores = cross_val_score(lin_reg, X_reg, y_reg, cv=5, scoring='r2')
```

```
print("cv_acc_scores: ", cv_acc_scores)
print("cv_f1_scores: ", cv_f1_scores)
print("cv_mse_scores: ", cv_mse_scores)
print("cv_r2_scores: ", cv_r2_scores)
```

```
cv_acc_scores: [0.93859649 0.94736842 0.98245614 0.92982456 0.96460177]
cv_f1_scores: [0.95302013 0.95833333 0.98611111 0.94444444 0.97142857]
cv_mse_scores: [0.48485857 0.62249739 0.64621047 0.5431996 0.49468484]
cv_r2_scores: [0.54866323 0.46820691 0.55078434 0.53698703 0.66051406]
```

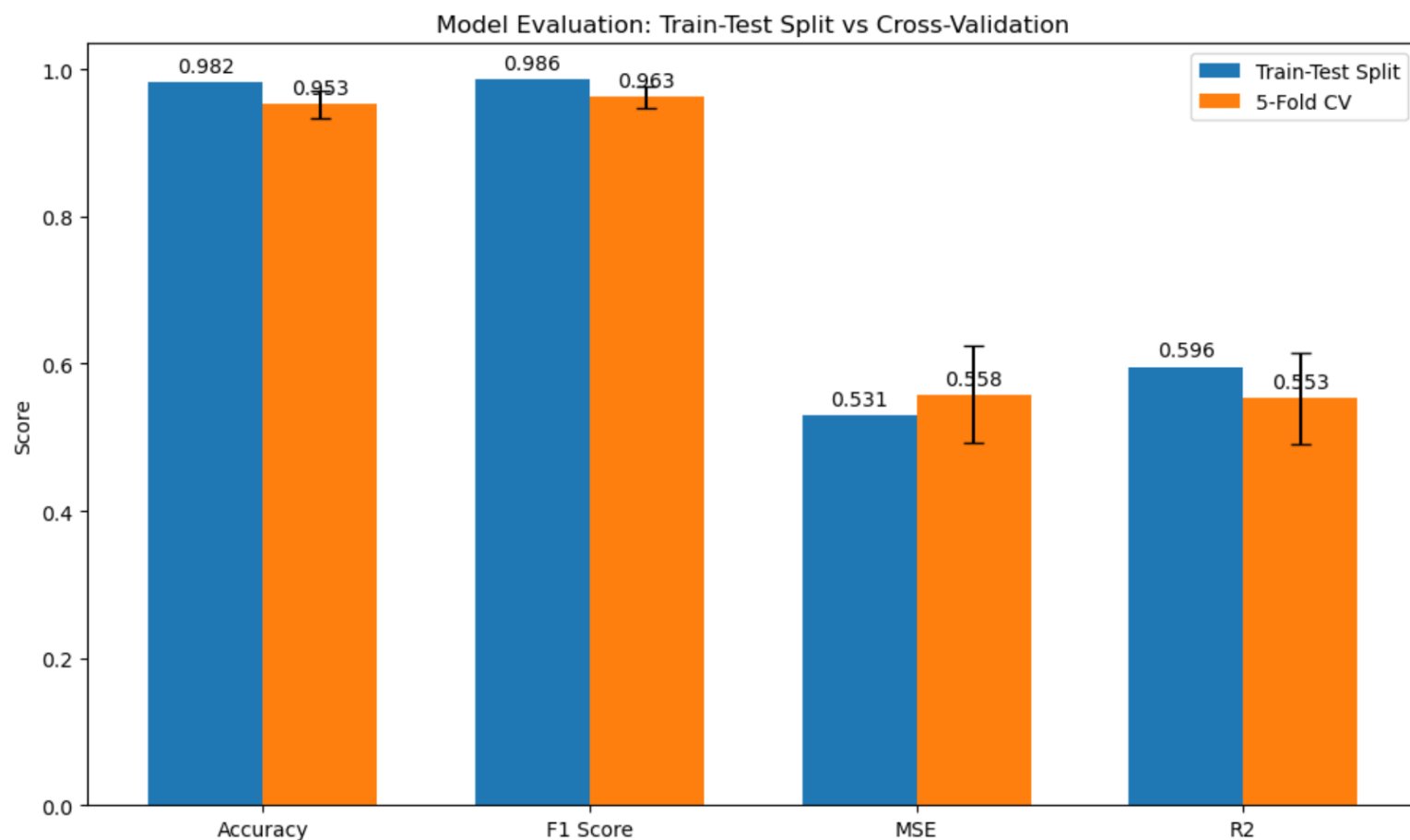
- Report the mean and standard deviation of the evaluation metrics.

```
cv_acc = np.mean(cv_acc_scores)
cv_f1 = np.mean(cv_f1_scores)
cv_mse = np.mean(cv_mse_scores)
cv_r2 = np.mean(cv_r2_scores)

# Standard deviations
cv_acc_std = np.std(cv_acc_scores)
cv_f1_std = np.std(cv_f1_scores)
cv_mse_std = np.std(cv_mse_scores)
cv_r2_std = np.std(cv_r2_scores)
```

c. Image Display Requirement:

- Display a visualization comparing the evaluation metrics obtained from the naive train-test split and k -fold cross-validation.
- The image must clearly label the metrics (Accuracy, F1 Score, MSE, R^2) and highlight performance differences.



```

# c. Visualization comparing metrics
import matplotlib.pyplot as plt
import seaborn as sns
metrics = ['Accuracy', 'F1 Score', 'MSE', 'R2']
train_test_values = [acc, f1, mse, r2]
cv_values = [cv_acc, cv_f1, cv_mse, cv_r2]
cv_errors = [cv_acc_std, cv_f1_std, cv_mse_std, cv_r2_std]

x = np.arange(len(metrics))
width = 0.35

fig, ax = plt.subplots(figsize=(10, 6))
rects1 = ax.bar(x - width/2, train_test_values, width, label='Train-Test Split')
rects2 = ax.bar(x + width/2, cv_values, width, yerr=cv_errors,
               capsize=5, label='5-Fold CV')

ax.set_ylabel('Score')
ax.set_title('Model Evaluation: Train-Test Split vs Cross-Validation')
ax.set_xticks(x)
ax.set_xticklabels(metrics)
ax.legend()

# Add values on top of bars
def autolabel(rects):
    for rect in rects:
        height = rect.get_height()
        ax.annotate(f'{height:.3f}',
                    xy=(rect.get_x() + rect.get_width() / 2, height),
                    xytext=(0, 3),
                    textcoords="offset points",
                    ha='center', va='bottom')

autolabel(rects1)
autolabel(rects2)

plt.tight_layout()
plt.show()

```

Task 2: Hyperparameter Tuning with GridSearchCV (or RandomizedSearchCV)

a. Logistic Regression (Breast Cancer Dataset):

- Define a parameter grid with the following suggested hyperparameters:
 - `C` (inverse regularization strength): $\{0.01, 0.1, 1, 10, 100\}$
 - `penalty`: $\{'l1', 'l2'\}$
 - Solver options (choose according to the selected penalty, e.g., `liblinear` or `saga`)

```
from sklearn.model_selection import GridSearchCV
```

```
param_grid_log = {  
    'C': [0.01, 0.1, 1, 10, 100],  
    'penalty': ['l1', 'l2'],  
    'solver': ['liblinear', 'saga']  
}
```

```
grid_log = GridSearchCV(  
    LogisticRegression(max_iter=5000),  
    param_grid_log,  
    cv=5,  
    scoring='accuracy',  
    n_jobs=-1  
)  
grid_log.fit(X_class, y_class)
```

```
best_log_params = grid_log.best_params_  
best_log_score = grid_log.best_score_  
print("best parameters: ", best_log_params)  
print("best score: ", best_log_score)
```

```
best parameters: {'C': 100, 'penalty': 'l1', 'solver': 'liblinear'}  
best score: 0.968390001552554
```

Task 2: Hyperparameter Tuning with GridSearchCV (or RandomizedSearchCV)

b. Linear Regression (California Housing Dataset):

- Compare plain Linear Regression with regularized versions (Ridge or Lasso Regression). For these models, consider tuning:
 - Regularization strength α : $\{0.001, 0.01, 0.1, 1, 10\}$

Standard linear regression tries to minimize the residual sum of squares:

$$\min_{\beta} \sum (y_i - X_i\beta)^2$$

- Goal: fit the data as closely as possible
- Problem: can **overfit**, especially with many features or multicollinearity

Ridge adds a penalty on the **squared magnitude** of coefficients:

$$\min_{\beta} \sum (y_i - X_i\beta)^2 + \lambda \sum \beta_j^2$$

Key idea:

- Penalizes **large coefficients**
- Shrinks them toward zero, but **never exactly zero**

Effects:

- Reduces overfitting
- Handles multicollinearity well
- Keeps all features (no feature selection)

Lasso adds a penalty on the **absolute value** of coefficients:

$$\min_{\beta} \sum (y_i - X_i\beta)^2 + \lambda \sum |\beta_j|$$

Key idea:

- Forces some coefficients to become **exactly zero**

Effects:

- Performs **feature selection**
- Produces sparse models (simpler, more interpretable)

```
from sklearn.linear_model import LinearRegression, Ridge, Lasso
plain_lr = LinearRegression()
plain_lr_score = np.mean(cross_val_score(plain_lr, X_reg, y_reg, cv=5, scoring='r2'))
```

```
param_grid_ridge = {'alpha': [0.001, 0.01, 0.1, 1, 10]}
param_grid_lasso = {'alpha': [0.001, 0.01, 0.1, 1, 10]}

grid_ridge = GridSearchCV(
    Ridge(),
    param_grid_ridge,
    cv=5,
    scoring='r2',
    n_jobs=-1
)
grid_ridge.fit(X_reg, y_reg)

grid_lasso = GridSearchCV(
    Lasso(max_iter=5000),
    param_grid_lasso,
    cv=5,
    scoring='r2',
    n_jobs=-1
)
grid_lasso.fit(X_reg, y_reg)
```

```
best_ridge_params = grid_ridge.best_params_  
best_lasso_params = grid_lasso.best_params_  
best_ridge_score = grid_ridge.best_score_  
best_lasso_score = grid_lasso.best_score_  
print("best linear score: ", plain_lr_score)  
print("best ridge parameters: ", best_ridge_params)  
print("best ridge score: ", best_ridge_score)  
print("best lasso parameters: ", best_lasso_params)  
print("best lasso score: ", best_lasso_score)
```

```
best linear score: 0.5530311140279234  
best ridge parameters: {'alpha': 10}  
best ridge score: 0.5531322809309795  
best lasso parameters: {'alpha': 0.001}  
best lasso score: 0.5530610428431253
```

c. **Implementation:**

- Use `GridSearchCV` (or `RandomizedSearchCV`) with cross-validation to perform hyperparameter tuning.
- Report the best hyperparameters along with the corresponding evaluation metrics.

d. **Image Display Requirement:**

- Display a visualization (e.g., a heatmap or line chart) that shows the performance across different hyperparameter combinations.
- Clearly annotate the plot with the best hyperparameter setting and its corresponding performance.

```
ridge_results = pd.DataFrame(grid_ridge.cv_results_)
lasso_results = pd.DataFrame(grid_lasso.cv_results_)
```

ridge_results

	mean_fit_time	std_fit_time	mean_score_time	std_score_time	param_alpha	params	split0_test_score	split1_test_score	split2_test_score	split3_test_score
0	0.009284	0.000868	0.002013	0.000193	0.001	{'alpha': 0.001}	0.548663	0.468207	0.550784	0.548663
1	0.010042	0.000370	0.002102	0.000500	0.010	{'alpha': 0.01}	0.548664	0.468207	0.550784	0.548664
2	0.010706	0.000688	0.001897	0.000234	0.100	{'alpha': 0.1}	0.548676	0.468204	0.550784	0.548676
3	0.007650	0.000850	0.001099	0.000238	1.000	{'alpha': 1}	0.548786	0.468173	0.550785	0.548786
4	0.004123	0.000426	0.001197	0.000152	10.000	{'alpha': 10}	0.549849	0.467875	0.550785	0.549849

Task 3: Comparison and Analysis

a. Evaluation Metrics:

- **Classification (Breast Cancer Dataset):** Accuracy and F1 Score.
- **Regression (California Housing Dataset):** Mean Squared Error (MSE) and R^2 .

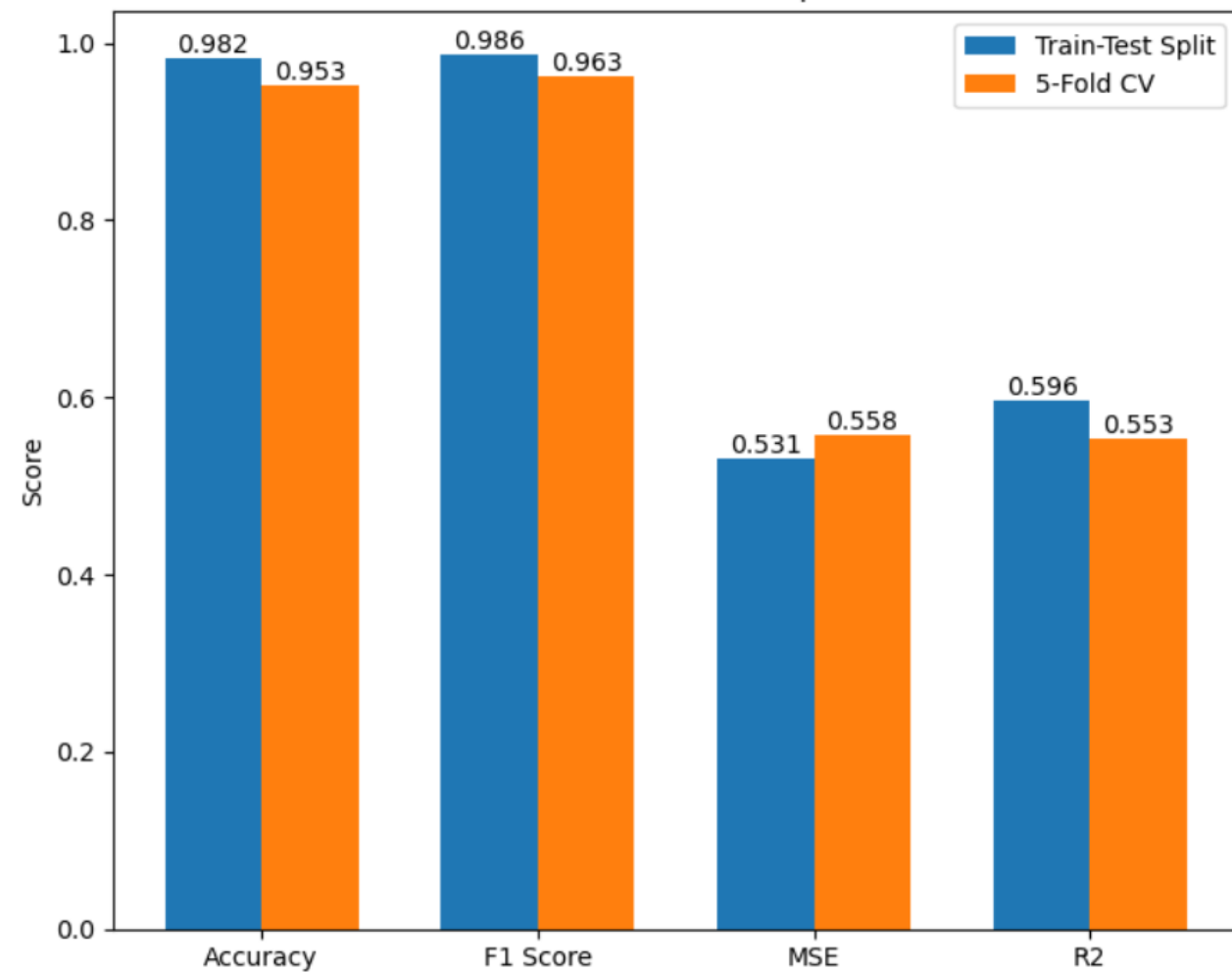
b. Comparative Visualization Requirement:

- Display visualizations that compare the evaluation metrics between the naive train-test split and k -fold cross-validation.
- Visualizations should clearly highlight the differences in performance (e.g., using side-by-side bar plots or overlaid line charts).

c. Analysis:

- Discuss how different data splitting strategies affect the evaluation metrics and model stability.

Evaluation Metrics Comparison



Cross-Validation Stability (Classification)

